

Programação Orientada a Objetos em Java

Eduardo Kessler Piveta

Universidade Federal de Santa Maria - UFSM

- Java é uma linguagem orientada a objetos amplamente usada no desenvolvimento de software.
- Java fornece suporte aos principais conceitos de orientação de objetos, incluindo: classes, atributos, métodos, herança, interfaces.
- Descreveremos como esses conceitos são disponibilizados pela linguagem.

Classes

- Em Java, uma classe pode ser definida através de uma declaração de classe contendo a palavra reservada `class`, um nome e um corpo (onde são definidos atributos e métodos).
- A seguir, um exemplo de uma classe `Conta`.

```
1 public class Conta{  
2     // Corpo da classe  
3 }
```

Atributos e Métodos

- No corpo da classe, podemos adicionar atributos e métodos.
- No exemplo a seguir, adicionamos um atributo saldo do tipo Double (linha 2), e um método sacar (linhas 3-5), que não retorna nada (void) e que tem um parâmetro formal chamado valor do tipo Double.

```
1 public class Conta{  
2     public Double saldo;  
3     public void sacar(Double valor){  
4         saldo = saldo - valor;  
5     }  
6 }
```

Atributos e Métodos

Considere, por exemplo, uma classe chamada `ContaCliente` que exercita algumas operações possíveis em uma conta.

```
1 public class ContaCliente {  
2     public static void main(String... args) {  
3         Conta c = new Conta();  
4         c.saldo = 100.00;  
5         c.sacar(50.00);  
6         c.depositar(100.00);  
7     }  
8 }
```

Note que os métodos `sacar` e `depositar` precisam de um objeto da classe `Conta` para serem invocados, pois são métodos de instância. Já o método `main` é um método de classe, o que faz com que ele não precise de uma instância de `ContaCliente` caso precisasse ser chamado por outra classe.

Encapsulamento

- O método `main` de `ContaCliente` acessa diretamente o saldo da `Conta` e faz com que o acoplamento entre as classes seja forte (o que não é recomendável).
- Uma alternativa é encapsular o acesso ao atributo `saldo`, fazendo com que todas as leituras e escritas sejam feitas através de métodos específicos para tal.
- Em Java, por exemplo, existem os seguintes modificadores de visibilidade:
 - padrão (visível dentro do pacote)
 - privado (visível apenas dentro da classe)
 - público (visível por todas as classes)
 - protegido (visível na classe e em todas as suas subclasses)

Encapsulamento - Exemplo

O código a seguir mostra as modificações feitas de forma a melhorar o encapsulamento do saldo e ocultar informações internas da classe de seus clientes.

```
1  public class Conta{
2      private Double saldo; // era public Double saldo;
3      public Double getSaldo(){
4          return saldo;
5      }
6      public void setSaldo(Double saldo){
7          this.saldo = saldo;
8      }
9      public void sacar(Double valor){
10         setSaldo(getSaldo() - valor); // era saldo = saldo
            - valor;
11     }
12 }
```

Encapsulamento - Exemplo

```
1 public class ContaCliente {
2     public static void main(String... args) {
3         Conta c = new Conta();
4         c.setSaldo(100.00); // Era c.saldo = 100.00;
5         c.sacar(50.00);
6         c.depositar(100.00);
7     }
8 }
```

Construtores

Em Java as classes possuem um construtor padrão, sem parâmetros, que é herdado da classe `Object`. Podem ser definidos novos construtores para facilitar o uso da classe por parte dos clientes.

```
1 public class Conta{
2     // Atributos e métodos de Conta
3     public Conta(Double valor){
4         setSaldo(valor);
5     }
6 }
```

Construtores

A classe ContaCliente poderia ser modificada de forma a usar o novo construtor definido (linha 3), passando o saldo inicial como parâmetro.

```
1 public class ContaCliente {  
2     public static void main(String... args) {  
3         Conta c = new Conta(100.00);  
4         c.sacar(50.00);  
5         c.depositar(100.00);  
6     }  
7 }
```

Em Java não há destrutores.

Herança

- Java permite a definição de herança simples para as classes (há herança múltipla apenas para interfaces) e as subclasses de uma determinada classe são subtipos da mesma.
- A especificação da herança é feita na subclasse através da palavra reservada `extends`: `Classe extends SuperClasse`.
- Classes que não contenham a definição de uma superclasse são implicitamente definidas como sendo subclasses da classe `Object` (a classe raiz da hierarquia de classes de Java).

Herança - Exemplo

```
1  public class ContaCorrente extends Conta{
2      private Double limiteDiarioSaque;
3      public Double getLimiteDiarioSaque(){
4          return limiteDiarioSaque;
5      }
6      public void setLimiteDiarioSaque(Double
           limiteDiarioSaque){
7          this.limiteDiarioSaque = limiteDiarioSaque;
8      }
9      // Outros construtores, atributos e métodos.
10 }
```

Herança - Sobrescrita

Nas subclasses, é comum termos **métodos sobrescritos**, em que um método definido na superclasse é estendido ou substituído por um método na subclasse.

```
1 public class ContaCorrente extends Conta{
2     public void sacar(Double valor){
3         super(valor);
4         calcularTarifaSaque(valor);
5     }
6     // Outros construtores, atributos e métodos.
7 }
```

Herança - Substituição de Liskov

- Pelo princípio de substituição de Liskov, podemos atribuir objetos de uma subclasse sempre que for esperado um objeto da superclasse.
 - Todo objeto de uma classe também é um objeto de sua superclasse - toda instância de ContaCorrente é também uma instância de Conta
- O contrário nem sempre é verdadeiro (nem toda Conta é uma ContaCorrente - poderia ser uma ContaPoupança, por exemplo).
 - Nesses casos, é necessário fazer uma conversão explícita (*cast*).

Herança - Substituição de Liskov

Conversão OK.

```
1  ContaCorrente cc = new ContaCorrente();  
2  Conta c = cc; // cc é uma ContaCorrente, que é uma  
   Conta.  
3  cc = (ContaCorrente) c; // Conversão explícita, ok.
```

Herança - Substituição de Liskov

Conversão com erro.

```
1  Conta c = new ContaCorrente();  
2  ContaPoupança cp = (ContaPoupança) c; // Conversão  
    explícita, erro de conversão.
```

Herança - Classes abstratas

Dado que a classe Conta possui diferentes subtipos, tais como ContaCorrente e ContaPoupança, além de outros em potencial (ContaInvestimento, por exemplo), podemos fazer com que apenas instâncias das subclasses possam ser criadas tornando a classe Conta abstrata (linha 1):

```
1  public abstract class Conta{  
2      // Atributos e métodos comuns a todos os tipos de  
        conta.  
3  }
```

Polimorfismo ad hoc (ou de sobrecarga)

Em relação ao polimorfismo, vamos ver alguns exemplos em Java. Em Java, o polimorfismo *ad-hoc* (ou de sobrecarga) é expresso através da definição de métodos de um mesmo nome com assinaturas diferentes.

```
1 public class Figura {  
2     public void redimensionar(Double percentual){  
3         // Código para redimensionar a figura por  
4             percentual  
5     }  
6     public void redimensionar(Integer x, Integer y){  
7         // Código para redimensionar a figura por  
8             coordenadas  
9     }  
10 }
```

Polimorfismo Paramétrico

Em relação ao polimorfismo paramétrico, podemos criar classes parametrizadas, através dos sinais < e >.

```
1  public class ArrayList<E> /* ... herança e interfaces
    implementadas */{
2      public boolean add(E e){
3          // Implementação de add
4      }
5      // Outros métodos e atributos da classe
6  }
```

Polimorfismo Paramétrico

Caso seja passado um objeto de um tipo que não seja uma String é gerado um erro em tempo de compilação por parte do sistema de tipos:

```
1  ArrayList<String> c = new ArrayList<>();  
2  c.add("Maria");  
3  c.add("João");  
4  ...
```

Polimorfismo Paramétrico

Caso quiséssemos uma lista de inteiros, poderíamos usar esse tipo de polimorfismo passando outro tipo para ArrayList:

```
1 ArrayList<Integer> c = new ArrayList<>();  
2 c.add(10);  
3 c.add(20);  
4 ...
```

Polimorfismo de subtipo

Retângulo: A seguir temos um exemplo:

```
1  Figura figura = new Círculo(10);
2  figura.desenhar(); // Aqui é o desenhar de Círculo que
   é chamado
3  figura = new Triângulo(20);
4  figura.desenhar(); // Aqui é o desenhar de Triângulo
5  figura = new Retângulo(10, 20);
6  figura.desenhar(); // Aqui é o desenhar de Retângulo
7  ...
```

Interfaces

- Em Java, as interfaces são definidas de maneira similar às classes, usando a palavra chave `interface`.
- Normalmente, as interfaces não contém atributos ou métodos, embora recentemente tenha sido adicionado suporte para métodos padrão (*default*), que já tem uma implementação pré-definida na interface.
- Para interfaces é permitida a herança múltipla (a interface `List`, que representa listas em Java, por exemplo, estende as interfaces `Collection` e `Iterable`

Interfaces

O exemplo a seguir mostra a definição de uma interface chamada IConta.

```
1  public interface IConta{  
2      public void sacar(Double valor);  
3      public void transferir(Conta destino, Double valor);  
4  }
```

Interfaces

Por exemplo, poderíamos ter uma classe que representa uma agência bancária, chamada Agência, que contém uma coleção de contas (especificada pela interface Collection com o parâmetro de tipos IConta):

```
1 public class Agência{  
2     Collection<IConta> contas;  
3     // ...  
4 }
```

Interfaces

Nesse caso, o cliente que usa a classe Agência pode escolher diferentes implementações de Collection

```
1 // Lista baseada em um vetor de contas correntes
2 Agência a1 = new Agência();
3 a1.setContas(new ArrayList<ContaCorrente>());
4 // Lista encadeada de poupanças
5 Agência a2 = new Agência();
6 a2.setContas(new LinkedList<ContaPoupança>());
```

Interfaces

- Para que seja possível passar as classes ContaCorrente e ContaPoupança como parâmetros de tipo para a coleção de contas de uma agência, tais classes precisam implementar a interface IConta.
- No entanto, não é necessário implementar os métodos de IConta nessas classes, pois ambas herdam da classe abstrata Conta, definida anteriormente.

```
1 public abstract class Conta implements IConta{
2     public void sacar(Double valor){
3         // Implementação concreta de sacar
4     }
5     public void transferir(Conta destino, Double valor){
6         // Implementação concreta de transferir
7     }
8 }
```

Conclusões

- Java é uma linguagem de programação amplamente utilizada e possui suporte aos principais conceitos de orientação a objetos.
- Em Java, são criadas classes, atributos, métodos e construtores para representar as principais abstrações do domínio de aplicação que se deseja.
- As classes podem ser compostas através de herança e relacionamentos (agregação, associação, etc), bem como através do uso de interfaces.
- O uso de herança possibilita o aproveitamento do polimorfismo de diferentes maneiras, facilitando o reuso de código.

Programação Orientada a Objetos em Java

Eduardo Kessler Piveta

Universidade Federal de Santa Maria - UFSM